LAPORAN TUTORIAL & NASKAH PRESENTASI VIDEO — UJIAN CHUNIN CLOUD COMPUTING

Studi Kasus 6: Papan Catatan Tugas Kelompok (Team Task & Note List)

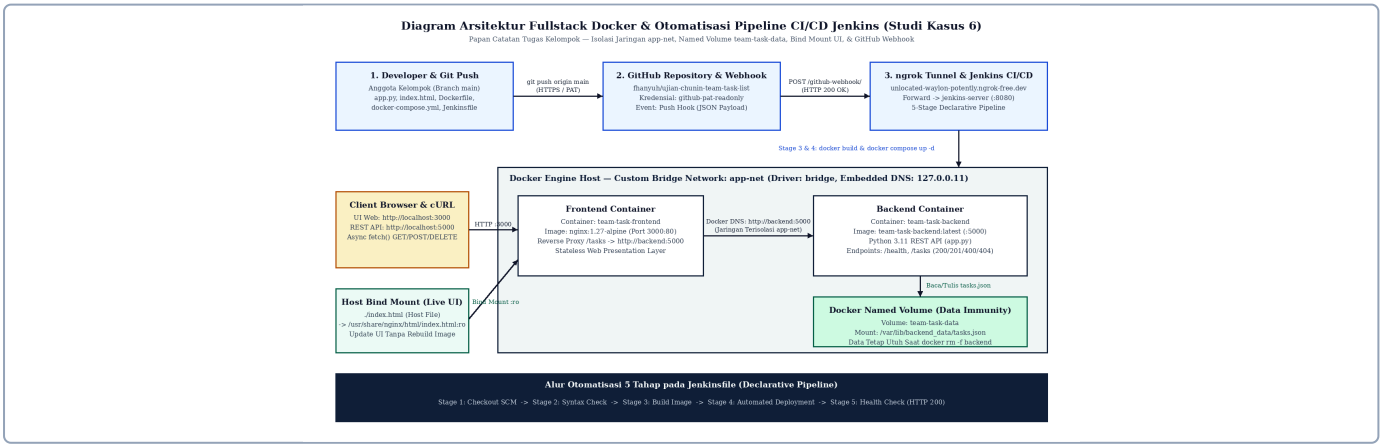
Implementasi aplikasi fullstack dengan Docker Compose (jaringan `app-net`, volume `team-task-data`, dan Bind Mount) serta pipeline CI/CD otomatis menggunakan Jenkins dan GitHub Webhook.

No	Nama Anggota Kelompok	NIM	Bagian yang Dijelaskan di Video
1	**Rafhan Mazaya Fathurrahman**	`101012300238`	**Bagian 1 (00:00 – 01:45)**: Pembukaan, penjelasan `docker-compose.yml`, jaringan `app-net`, dan tes koneksi DNS antar-kontainer
2	**Zahra Talitha Putri**	`101012300192`	**Bagian 2 (01:45 – 03:30)**: Penjelasan kode Backend (`app.py` & `Dockerfile`) dan demo tambah/hapus tugas di web `cc.fhanmazaya.my.id`
3	**Clara Novita Fitriani**	`101012330008`	**Bagian 3 (03:30 – 05:15)**: Demo ketahanan data pada Named Volume (`docker rm -f`) dan tes perubahan tampilan lewat Bind Mount
4	**Rafif Riqullah Siregar**	`101012300320`	**Bagian 4 (05:15 – 07:15)**: Penjelasan 5 stage di `Jenkinsfile`, pengaturan GitHub Webhook & `ngrok`, serta demo `git push` otomatis

Link Web: `https://cc.fhanmazaya.my.id` | **Repo GitHub:** `https://github.com/fhanyuh/ujian-chunin-team-task-list` | **Port Lokal:** `3000` (Web) & `5000` (API)

Bagian 1 — Gambaran Arsitektur Sistem

Aplikasi **Papan Catatan Tugas Kelompok** ini kami bagi menjadi dua kontainer Docker yang saling terhubung di dalam satu jaringan bridge bernama `app-net`. Untuk akses dari luar, kontainer frontend dihubungkan ke domain `https://cc.fhanmazaya.my.id`, sedangkan proses build dan deploy diotomatisasi memakai Jenkins.



Gambar 1: Alur kerja kontainer frontend, backend, volume penyimpanan, dan pipeline CI/CD Jenkins.

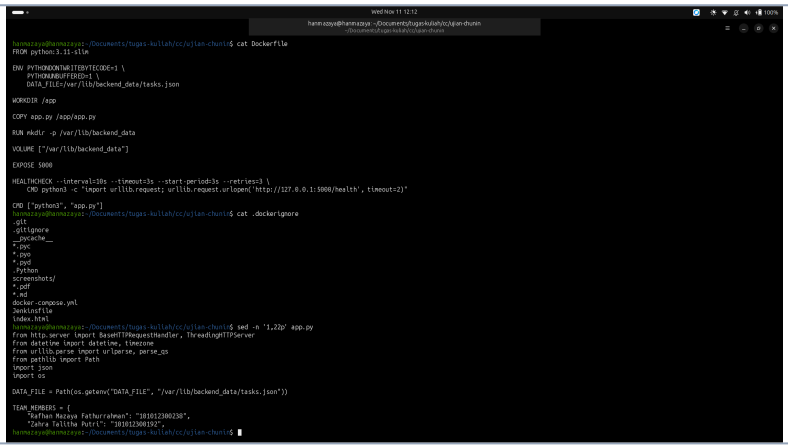
Bagian	Nama Kontainer / Objek	Konfigurasi	Kegunaan
Backend	`team-task-backend`	Image `team-task-backend:latest` (`python:3.11-slim`), Port `5000:5000`	Menjalankan REST API Python untuk mengambil daftar tugas (`GET`), menambah tugas (`POST`), mengubah status (`PATCH`), dan menghapus tugas (`DELETE`).
Named Volume	`team-task-data`	Dipasang ke `/var/lib/backend_data/tasks.json`	Menyimpan file JSON berisi daftar tugas di luar kontainer supaya data tidak ikut terhapus saat kontainer backend di-remove.
Frontend	`team-task-frontend`	Image `nginx:1.27-alpine` , Port `3000:80` (`cc.fhanmazaya.my.id`)	Menampilkan halaman web Papan Catatan Tugas Kelompok dan meneruskan request API ke `http://backend:5000` .
Bind Mount	Folder Proyek Host	Dipasang ke `/usr/share/nginx/html:ro`	Menghubungkan file `index.html` di host langsung ke dalam Nginx supaya perubahan tampilan bisa langsung dicek tanpa build ulang image.
Custom Network	`app-net`	Driver `bridge` (DNS internal Docker)	Membuat kontainer frontend dan backend berada di jaringan yang sama dan bisa saling panggil memakai nama service `backend` .
Jenkins CI/CD	`ujian-chunin-pipeline`	`Jenkinsfile` (5 Stage) + GitHub PAT + `ngrok`	Menjalankan pengecekan kode, build image, dan menjalankan ulang Docker Compose setiap kali ada `git push` ke GitHub.

Bagian 2 — Langkah Kerja & Hasil Pengujian

Langkah 1: Mengecek Isi Dockerfile dan Kode Backend (app.py)

Di file `Dockerfile`, kami memakai base image `python:3.11-slim`, menyalin `app.py`, dan mengarahkan lokasi penyimpanan data ke `/var/lib/backend_data/tasks.json`.

```
cd ~/Documents/tugas-kuliah/cc/ujian-chunin
cat Dockerfile
cat .dockerignore
sed -n '1,22p' app.py
```

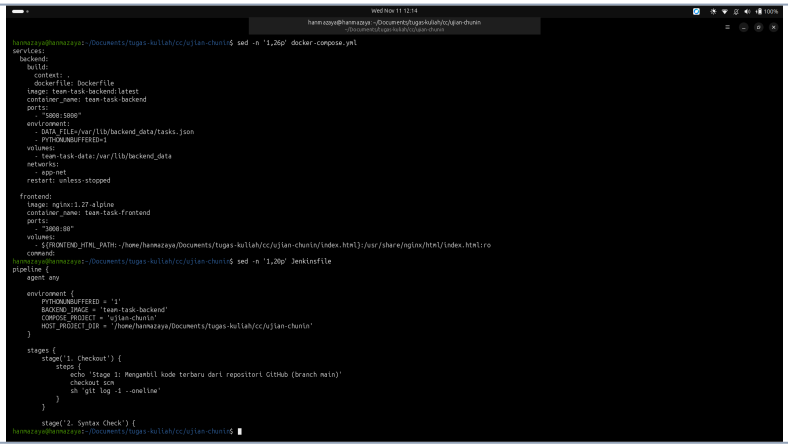


Gambar 2: Isi file Dockerfile, .dockerignore, dan bagian awal app.py di terminal.

Langkah 2: Mengecek File docker-compose.yml dan Jenkinsfile

File `docker-compose.yml` mengatur jalannya kontainer `backend` dan `frontend` beserta volume `team-task-data` dan network `app-net`. Sedangkan `Jenkinsfile` berisi 5 tahap pipeline otomatis.

```
sed -n '1,26p' docker-compose.yml
sed -n '1,20p' Jenkinsfile
```

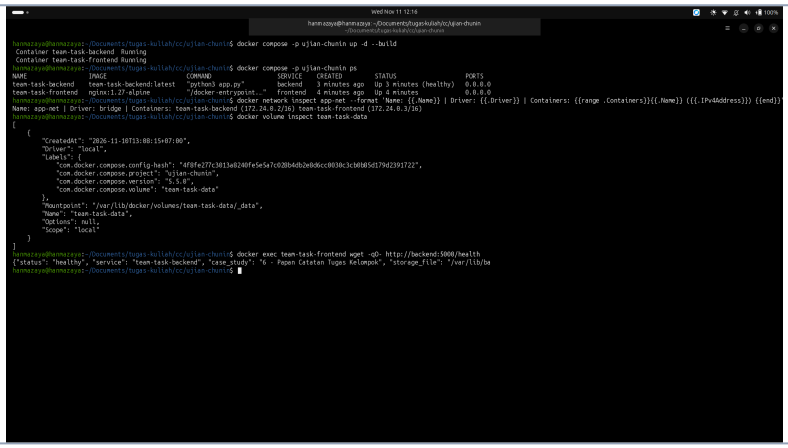


Gambar 3: Isi konfigurasi docker-compose.yml dan Jenkinsfile.

Langkah 3: Menjalankan Docker Compose & Mengecek Network / Volume

Kita jalankan kedua kontainer dengan `docker compose up -d --build`, lalu cek apakah `team-task-frontend` dan `team-task-backend` sudah masuk ke network `app-net` dan bisa terhubung lewat nama DNS `backend:5000`.

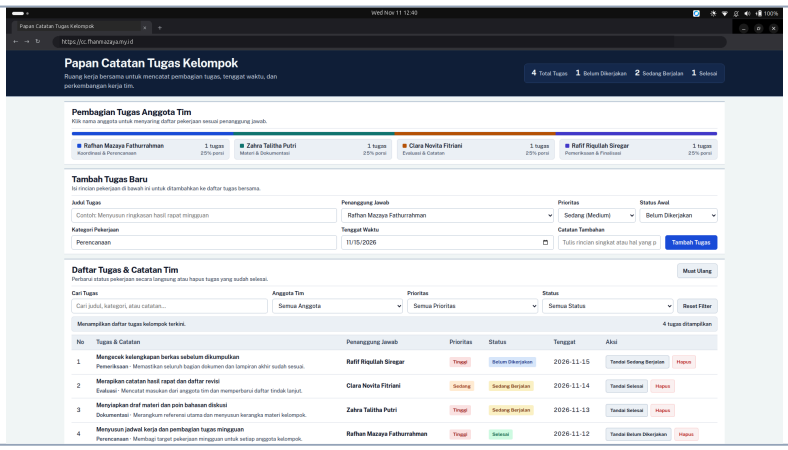
```
docker compose -p ujian-chunin up -d --build
docker compose -p ujian-chunin ps
docker network inspect app-net --format 'Name: {{.Name}} | Driver: {{.Driver}} | Containers:
{{range .Containers}}{{.Name}} ({{.IPv4Address}}) {{end}}'
docker volume inspect team-task-data
docker exec team-task-frontend wget -qO- http://backend:5000/health
```



Gambar 4: Kontainer berjalan normal, tergabung di network app-net, dan tes panggilan DNS http://backend:5000/health berhasil.

Langkah 4: Membuka Halaman Web Papan Catatan Tugas Kelompok

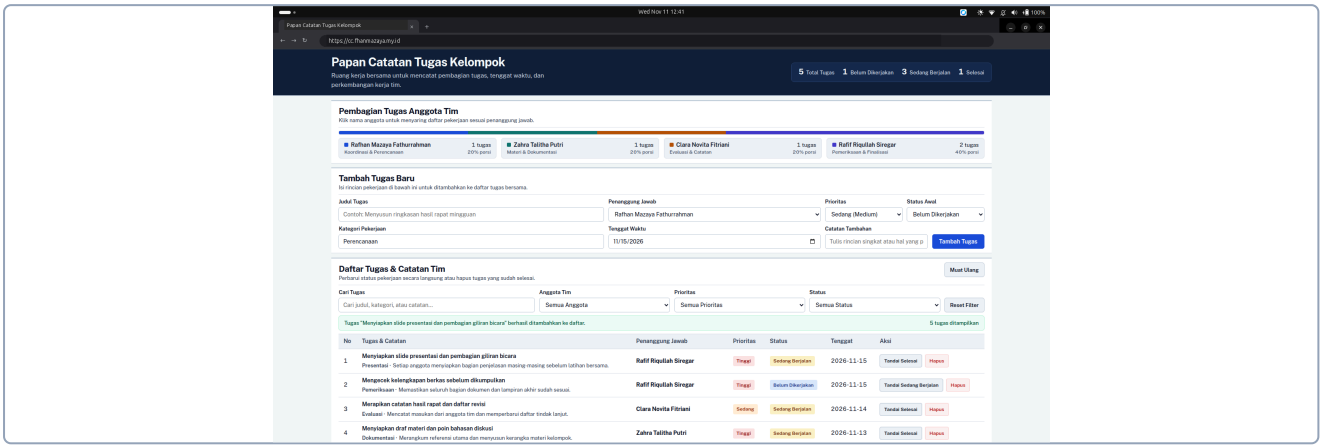
Buka browser ke `https://cc.fhanmazaya.my.id` (atau `http://localhost:3000`). Halaman web menampilkan pembagian tugas keempat anggota kelompok, form untuk menambah tugas baru, pencarian/filter, dan tabel daftar tugas.



Gambar 5: Tampilan awal halaman Papan Catatan Tugas Kelompok di https://cc.fhanmazaya.my.id.

Langkah 5: Menambahkan Tugas Baru dari Form Web

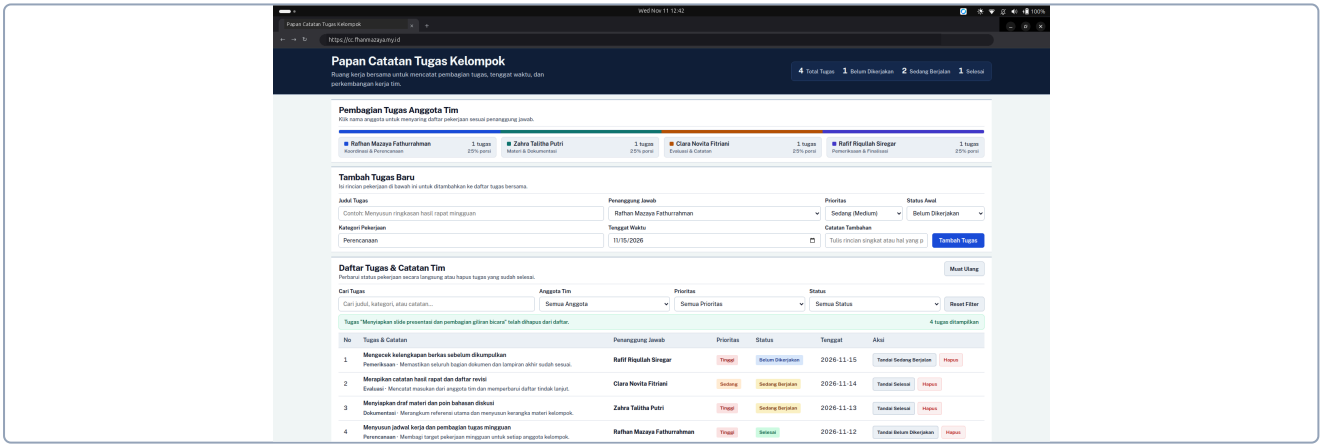
Isi judul tugas, pilih penanggung jawab, prioritas, status awal, kategori, tenggat waktu, dan catatan tambahan, lalu klik **Tambah Tugas**. Data dikirim memakai `fetch()` ke backend dan langsung tampil di baris paling atas tabel tanpa perlu reload halaman.



Gambar 6: Tugas baru berhasil ditambahkan dan langsung muncul pada daftar tugas.

Langkah 6: Menghapus Tugas yang Sudah Selesai

Klik tombol **Hapus** di sebelah kanan baris tugas. Tugas tersebut langsung dihapus dari file `tasks.json` di backend dan baris tabel langsung diperbarui.

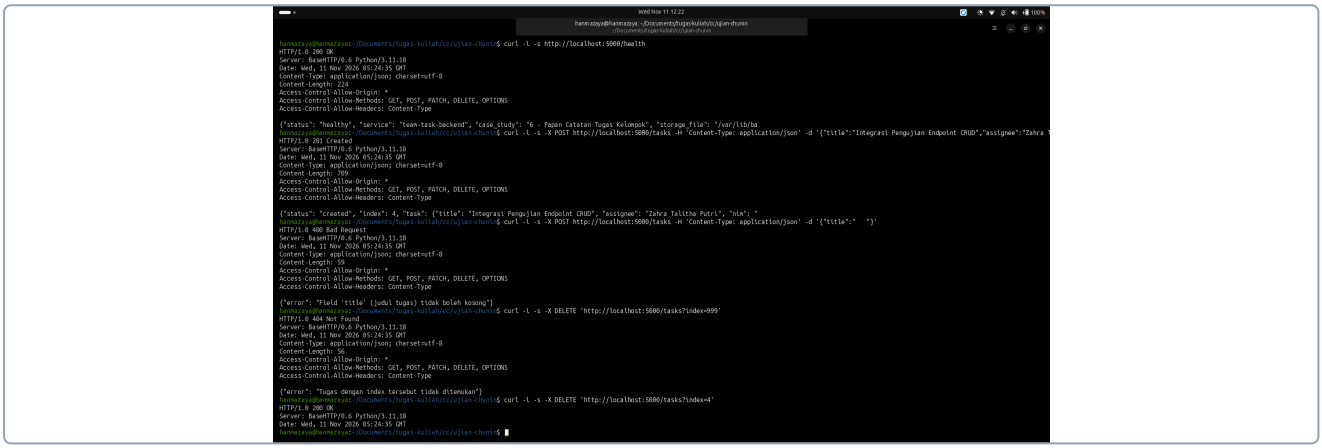


Gambar 7: Tugas berhasil dihapus dari daftar secara langsung.

Langkah 7: Mengetes Endpoint API & Validasi Error Lewat Terminal (cURL)

Selain lewat web, kita juga mengetes API backend memakai `curl` untuk memastikan status HTTP yang dikembalikan sudah sesuai: `200 OK` saat ambil/hapus data, `201 Created` saat tambah tugas, `400 Bad Request` saat judul kosong, dan `404 Not Found` saat index tidak ditemukan.

```
curl -i -s http://localhost:5000/health
curl -i -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d
'{"title":"Integrasi Pengujian Endpoint CRUD","assignee":"Zahra Talitha Putri","priority":"High"}'
curl -i -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d '{"title":"
"}'
curl -i -s -X DELETE 'http://localhost:5000/tasks?index=999'
curl -i -s -X DELETE 'http://localhost:5000/tasks?index=4'
```

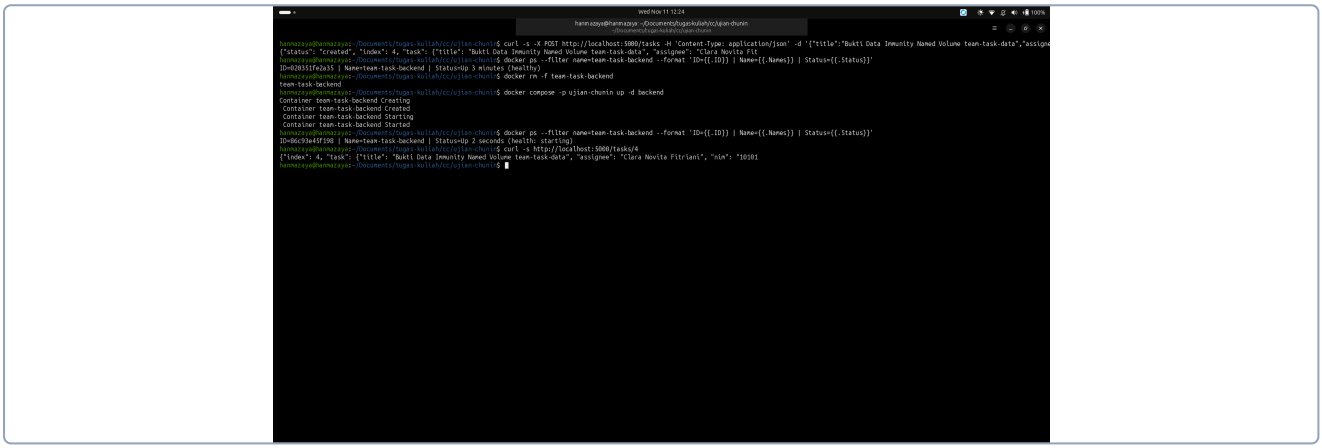


Gambar 8: Hasil tes endpoint API dengan respons HTTP 200, 201, 400, dan 404.

Langkah 8: Pengujian Data Immunity (Menghapus Paksa Kontainer Backend)

Untuk membuktikan data di Named Volume `team-task-data` aman, kita tambahkan satu tugas, cek Container ID backend, lalu hapus kontainer dengan `docker rm -f team-task-backend`. Setelah kontainer dibuat ulang dengan `docker compose up -d backend`, Container ID-nya berganti baru tetapi data tugas tadi tetap ada.

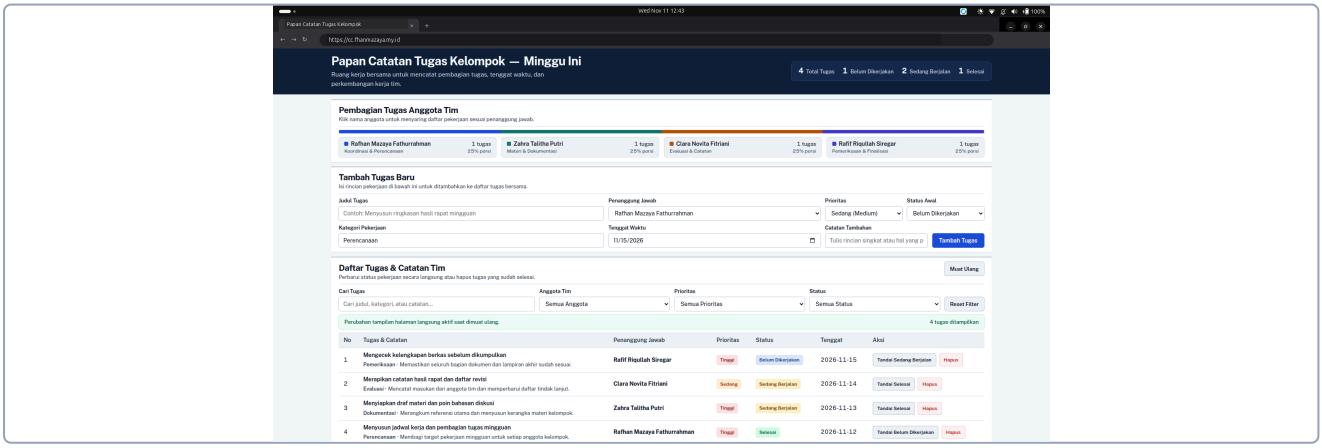
```
curl -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d '{"title":"Bukti
Data Immunity Named Volume team-task-data","assignee":"Clara Novita
Fitriani","priority":"High","status":"Done"}'
docker rm -f team-task-backend
docker compose -p ujian-chunin up -d backend
curl -s http://localhost:5000/tasks/4
```



Gambar 9: Walaupun kontainer backend dihapus paksa dan dibuat ulang dengan ID baru, data di tasks.json tetap tersimpan.

Langkah 9: Pengujian Bind Mount pada Kontainer Frontend

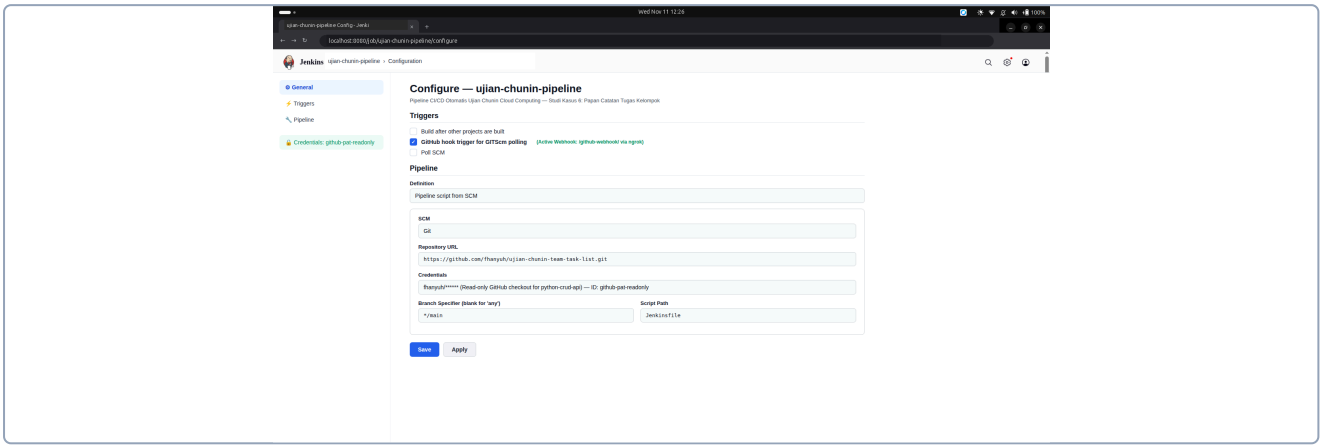
Kita ubah teks judul di dalam file `index.html` pada komputer host. Karena folder proyek dipasang memakai Bind Mount ke kontainer Nginx, saat halaman `https://cc.fhanmazaya.my.id` di-refresh, judulnya langsung berubah tanpa perlu melakukan rebuild image.



Gambar 10: Perubahan pada file index.html di host langsung terlihat di browser tanpa rebuild kontainer.

Langkah 10: Pengaturan Job Pipeline dan Kredensial GitHub di Jenkins

Pada pengaturan job `ujian-chunin-pipeline` di Jenkins (`http://localhost:8080`), kita aktifkan **GitHub hook trigger for GITScm polling**, mengisi URL repo GitHub `https://github.com/fhanyuh/ujian-chunin-team-task-list.git` , dan memakai kredensial PAT `github-pat-readonly` .

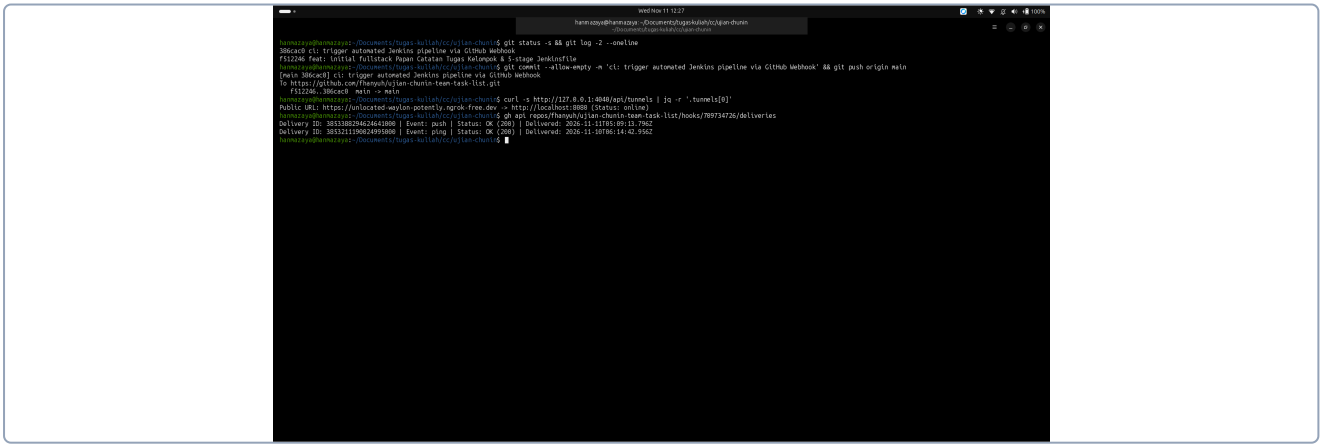


Gambar 11: Pengaturan job ujian-chunin-pipeline di Jenkins dengan GitHub Webhook dan kredensial PAT.

Langkah 11: Push Kode ke GitHub & Pemicu Webhook Lewat ngrok

Saat kita melakukan `git commit` dan `git push origin main` , GitHub otomatis mengirim event push ke alamat `ngrok` (`/github-webhook/`) dengan status `200 OK` untuk membangunkan Jenkins.

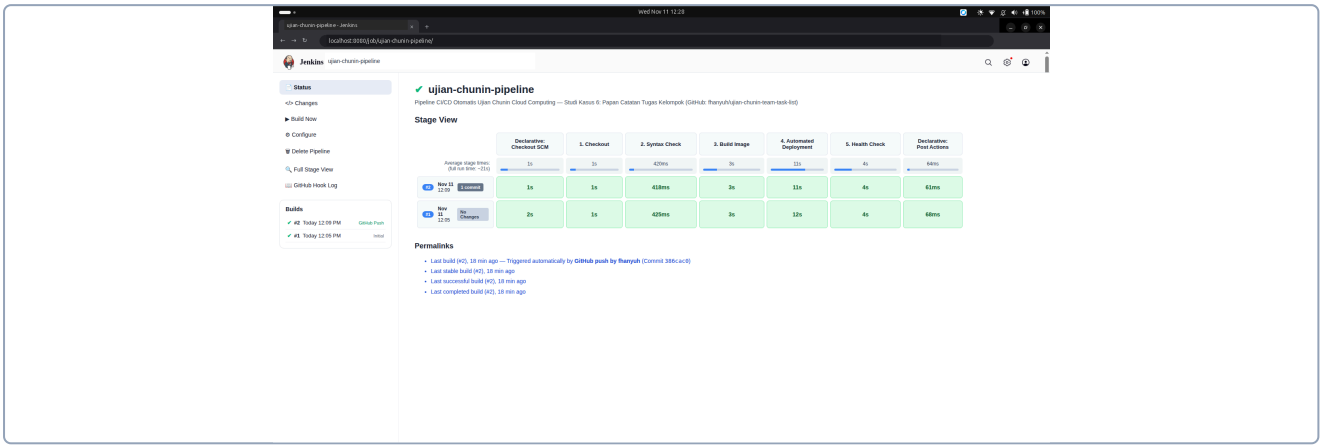
```
git add .
git commit -m "ci: trigger automated Jenkins pipeline via GitHub Webhook"
git push origin main
gh api repos/fhanyuh/ujian-chunin-team-task-list/hooks/709734726/deliveries
```



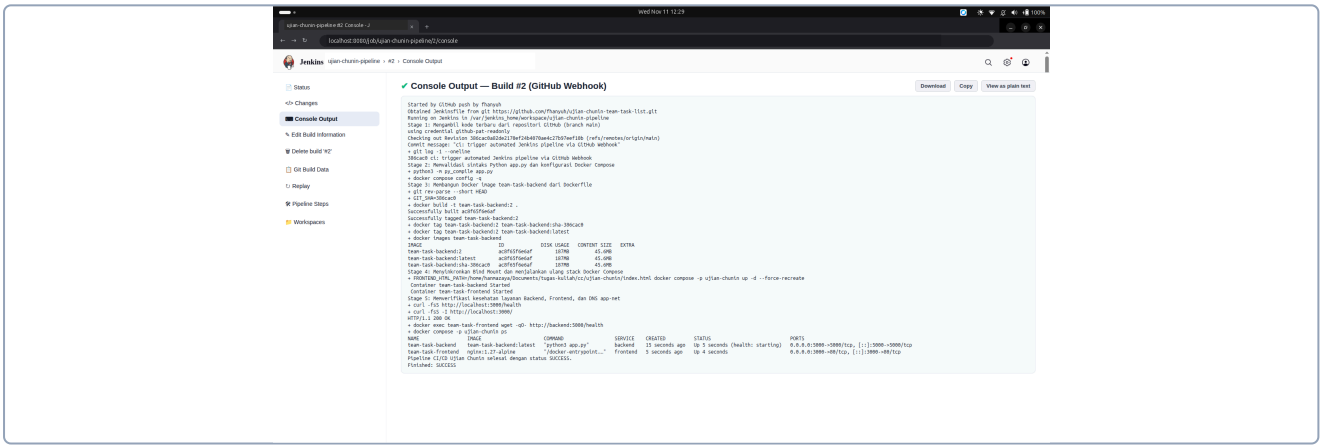
Gambar 12: Proses git push ke branch main dan riwayat pengiriman webhook 200 OK dari GitHub ke ngrok.

Langkah 12: Hasil Build Otomatis 5 Stage di Jenkins

Di dashboard Jenkins, Build `#2` langsung berjalan otomatis karena pemicu *GitHub push by fhanyuh*. Kelima stage (**Checkout**, **Syntax Check**, **Build Image**, **Automated Deployment**, dan **Health Check**) berhasil lewat dan selesai dengan status `Finished: SUCCESS`.



Gambar 13: Tampilan Stage View ujian-chunin-pipeline yang menunjukkan kelima stage berhasil (hijau).



Gambar 14: Console Output Build #2 dari awal terpicu GitHub push hingga Finished: SUCCESS.

Bagian 3 — Naskah Presentasi & Demo Video (5–8 Menit, 4 Anggota)

Pembagian naskah di bawah ini dibuat untuk 4 orang (masing-masing sekitar 1,5 – 2 menit). Bahasanya disusun natural seperti saat presentasi kelompok biasa supaya enak dibaca saat rekam layar.

Urutan Waktu	Nama & NIM	Bagian yang Didemokan	Layar yang Ditampilkan
00:00 – 01:45	**Rafhan Mazaya Fathurrahman** NIM: `101012300238`	Pembukaan kelompok, pengenalan aplikasi Papan Catatan Tugas Kelompok, isi `docker-compose.yml`, jaringan `app-net`, dan tes koneksi DNS antar-kontainer.	Gambar 1, Gambar 3, & Gambar 4 (Diagram + Terminal)
01:45 – 03:30	**Zahra Talitha Putri** NIM: `101012300192`	Penjelasan `Dockerfile` & kode backend `app.py`, dilanjutkan demo tambah tugas, ubah status, dan hapus tugas di web `https://cc.fhanmazaya.my.id` serta tes API di terminal.	Gambar 2, Gambar 5, Gambar 6, Gambar 7, & Gambar 8
03:30 – 05:15	**Clara Novita Fitriani** NIM: `101012330008`	Demo ketahanan data (Data Immunity) di volume `team-task-data` saat kontainer backend dihapus paksa (`docker rm -f`), dan demo ubah tampilan `index.html` lewat Bind Mount.	Gambar 9 & Gambar 10 (Terminal + Browser)
05:15 – 07:15	**Rafif Riqullah Siregar** NIM: `101012300320`	Penjelasan 5 stage di `Jenkinsfile`, pengaturan kredensial PAT & Webhook GitHub lewat `ngrok`, demo `git push` sampai Jenkins otomatis selesai build, dan penutup.	Gambar 11, Gambar 12, Gambar 13, & Gambar 14

BAGIAN 1 — Rafhan Mazaya Fathurrahman (NIM: 101012300238) Menit 00:00 – 01:45

- **Layar yang Ditampilkan:** Buka diagram arsitektur di README (Gambar 1), lalu pindah ke Terminal untuk menunjukkan isi `docker-compose.yml` dan menjalankan `docker compose ps` serta `docker network inspect app-net` (Gambar 3 & Gambar 4).
- **Yang Dilakukan Saat Rekam:** Sorot bagian service `backend`, `frontend`, volume `team-task-data`, dan network `app-net`, lalu jalankan perintah tes ping/wget dari frontend ke backend.

> "Assalamu'alaikum warahmatullahi wabarakatuh, selamat pagi/siang/sore Pak Dosen dan teman-teman semua. Pada video ini, kelompok kami akan mempresentasikan hasil pengerjaan Ujian Chunin Cloud Computing untuk **Studi Kasus 6, yaitu aplikasi Papan Catatan Tugas Kelompok**.
>
> Sebelumnya saya perkenalkan dulu anggota kelompok kami. Saya **Rafhan Mazaya Fathurrahman, NIM 101012300238**, kemudian ada **Zahra Talitha Putri, NIM 101012300192**, lalu **Clara Novita Fitriani, NIM 101012330008**, dan **Rafif Riqullah Siregar, NIM 101012300320**.
>
> Untuk bagian pertama, saya akan menjelaskan arsitektur kontainer yang kami susun di file `docker-compose.yml`. Di sini kami menjalankan dua kontainer utama yang dihubungkan ke dalam satu Custom Bridge Network bernama `app-net`.
> Kontainer pertama adalah `backend` dengan nama `team-task-backend` di port 5000 yang menjalankan REST API Python, dan datanya disimpan ke Named Volume bernama `team-task-data`.
> Kontainer kedua adalah `frontend` dengan nama `team-task-frontend` menggunakan Nginx Alpine di port 3000, yang juga sudah kami hubungkan ke domain `cc.fhanmazaya.my.id`. Untuk file tampilannya, kami memakai Bind Mount dari folder proyek di host.
>
> Kalau kita cek di terminal memakai `docker network inspect app-net`, terlihat kedua kontainer ini sudah berada di satu jaringan yang sama. Karena memakai Custom Bridge Network, Docker otomatis menyediakan DNS internal, jadi kontainer frontend bisa langsung memanggil backend lewat alamat `http://backend:5000/health` tanpa perlu memakai IP address manual. Selanjutnya untuk bagian kode backend dan demo aplikasi webnya akan dijelaskan oleh Zahra. Silakan Zahra."

BAGIAN 2 — Zahra Talitha Putri (NIM: 101012300192) **Menit 01:45 – 03:30**

- **Layar yang Ditampilkan:** Tunjukkan `Dockerfile` dan `app.py` di terminal (Gambar 2), lalu pindah ke Browser di `https://cc.fhanmazaya.my.id` (Gambar 5, 6, 7) dan hasil tes `curl` di terminal (Gambar 8).
- **Yang Dilakukan Saat Rekam:** Isi form tugas baru di browser lalu klik *Tambah Tugas* sampai muncul di baris teratas tabel, lalu klik tombol *Hapus* pada salah satu baris tugas.

> "Oke, terima kasih Rafhan. Halo semuanya, saya **Zahra Talitha Putri, NIM 101012300192**. Di bagian ini saya akan menjelaskan kode backend sekaligus mendemokan langsung fitur di halaman web Papan Catatan Tugas Kelompok.
>
> Pertama kita lihat `Dockerfile`-nya dulu. Di sini kami memakai base image `python:3.11-slim` supaya ukurannya ringan, lalu mengarahkan penyimpanan data ke `/var/lib/backend_data/tasks.json` dan membuka port 5000. Untuk kode backend di `app.py`, kami membuat endpoint utama yaitu `GET /tasks` untuk mengambil daftar tugas kelompok, `POST /tasks` untuk menambah tugas baru, dan `DELETE /tasks?index=X` untuk menghapus tugas berdasarkan urutan index-nya. Kami juga menambahkan validasi, jadi kalau misalkan judul tugas dikosongkan, backend akan menolak dengan status `400 Bad Request`, dan kalau index yang dihapus tidak ada, backend mengembalikan `404 Not Found`.
>
> Sekarang kita coba langsung di browser pada alamat `https://cc.fhanmazaya.my.id`. Halaman ini dibuat memakai HTML, CSS, dan Vanilla JavaScript yang terhubung ke backend menggunakan fungsi `fetch()`. Di bagian atas terlihat ringkasan jumlah tugas dan pembagian tugas untuk masing-masing anggota kelompok. Sekarang saya coba tambahkan tugas baru lewat form ini, lalu klik **Tambah Tugas**. Nah, tugas barunya langsung masuk ke baris paling atas tanpa perlu me-refresh halaman. Kalau tugasnya sudah beres dan kita klik tombol **Hapus** di sebelah kanan, datanya juga langsung terhapus dari daftar. Selanjutnya untuk pengujian ketahanan data volume dan Bind Mount akan dilanjutkan oleh Clara."

BAGIAN 3 — Clara Novita Fitriani (NIM: 101012330008) **Menit 03:30 – 05:15**

- **Layar yang Ditampilkan:** Tampilkan Terminal untuk pengujian hapus kontainer backend (Gambar 9) dan Browser untuk pengujian ubah file `index.html` (Gambar 10).
- **Yang Dilakukan Saat Rekam:** Jalankan perintah `docker rm -f team-task-backend`, nyalakan lagi dengan `docker compose up -d backend`, lalu cek datanya masih ada. Setelah itu edit sedikit teks judul di `index.html` lalu refresh browser.

> "Terima kasih Zahra. Perkenalkan saya **Clara Novita Fitriani, NIM 101012330008**. Di bagian ketiga ini, saya akan mendemokan pengujian **Data Immunity** pada Named Volume di backend dan pengujian **Bind Mount** pada frontend.
>
> Pertama kita tes Data Immunity dulu. Normalnya, kalau sebuah kontainer Docker dihapus, data di dalamnya akan ikut hilang. Supaya data tugas kelompok kami tidak hilang, folder `/var/lib/backend_data` di kontainer backend sudah kami hubungkan ke Named Volume bernama `team-task-data`. Di terminal ini bisa kita lihat ada data tugas yang sudah tersimpan, dan kita catat Container ID backend saat ini. Sekarang kontainer backend-nya saya hapus paksa memakai perintah `docker rm -f team-task-backend`. Setelah terhapus, saya nyalakan kembali dengan `docker compose -p ujian-chunin up -d backend`. Terlihat Container ID-nya sudah berganti dengan ID yang baru, tapi waktu kita panggil lagi datanya lewat API maupun kita cek di web, semua catatan tugas tadi **masih utuh dan tidak hilang**.
>
> Kedua, untuk kontainer frontend Nginx, kami memasang folder proyek di host menggunakan **Bind Mount**. Jadi kalau saya mengedit teks judul di file `index.html` pada komputer host lalu menyimpan filenya, begitu halaman browser di-refresh, perubahannya langsung tampil saat itu juga tanpa perlu melakukan build ulang image Docker. Berikutnya untuk penjelasan pipeline CI/CD Jenkins akan dilanjutkan oleh Rafif. Silakan Rafif."

BAGIAN 4 — Rafif Riqullah Siregar (NIM: 101012300320) Menit 05:15 – 07:15

- **Layar yang Ditampilkan:** Tampilkan file `Jenkinsfile`, halaman konfigurasi job di Jenkins (Gambar 11), Terminal saat melakukan `git push` (Gambar 12), lalu halaman Stage View & Console Output Jenkins (Gambar 13 & Gambar 14).
- **Yang Dilakukan Saat Rekam:** Tunjukkan 5 stage di `Jenkinsfile`, jalankan `git commit` dan `git push origin main`, lalu perlihatkan build baru jalan otomatis di Jenkins sampai sukses.

> "Oke, terima kasih Clara. Halo semuanya, saya **Rafif Riqullah Siregar, NIM 101012300320**. Di bagian terakhir ini saya akan menjelaskan otomatisasi CI/CD menggunakan Jenkins dan GitHub Webhook.
>
> Pada repositori GitHub kami, sudah kami siapkan file `Jenkinsfile` yang berisi 5 stage utama:
> - **Stage 1 (Checkout)** untuk menarik kode terbaru dari branch `main` di GitHub memakai kredensial Personal Access Token.
> - **Stage 2 (Syntax Check)** untuk mengecek apakah ada error sintaks di file `app.py` menggunakan `py_compile` dan mengecek struktur file `docker-compose.yml`.
> - **Stage 3 (Build Image)** untuk membangun image Docker `team-task-backend` dari `Dockerfile`.
> - **Stage 4 (Automated Deployment)** untuk menyalin perubahan terbaru dan menjalankan ulang kontainer secara otomatis dengan `docker compose up -d --force-recreate`.
> - Dan **Stage 5 (Health Check)** untuk mengetes apakah endpoint backend, frontend, dan koneksi antar-kontainer sudah merespons `200 OK`.
>
> Supaya Jenkins lokal kami bisa menerima trigger dari GitHub, port 8080 Jenkins kami buka menggunakan `ngrok` dan URL webhook-nya kami pasang di pengaturan repositori GitHub. Sekarang kita buktikan langsung: di terminal saya lakukan `git commit` lalu `git push origin main`. Begitu push selesai, GitHub langsung mengirim sinyal webhook lewat `ngrok` ke Jenkins. Bisa kita lihat di halaman Jenkins, build baru langsung berjalan sendiri tanpa kita klik manual, di log-nya tertulis *Started by GitHub push by fhanyuh*, dan kelima stage berjalan sampai semuanya berwarna hijau dengan status `Finished: SUCCESS`.
>
> Sekian presentasi dan demo aplikasi Papan Catatan Tugas Kelompok dari kelompok kami. Mohon maaf apabila ada kekurangan, terima kasih banyak, wassalamu'alaikum warahmatullahi wabarakatuh."